UPPSALA
UNIVERSITET

# Parallel and Distributed Programming

## Assignment-2

# Assignment-2(Parallelised sequential code using MPI)

**Submitted by**

Rajib Datta
rajib.datta.3822@student.uu.se

**Department of Information Technology**

April 17, 2026

# Contents

**1 Introduction** **3**

**2 Task 2.1: MPI Parallel Stencil Implementation** **3**
2.1 Data Distribution . . . 3
2.2 Computation Strategy . . . 3
2.3 Communication Pattern . . . 3
2.4 Time Measurement . . . 4
2.5 Summary . . . 4

**3 Task 2.2: Requirements for the Parallelised Code** **4**
3.1 Data Distribution Requirement . . . 4
3.2 Local Computation Requirement . . . 4
3.3 Communication Requirement (Halo Exchange) . . . 4
3.4 Iteration Requirement . . . 4
3.5 Result Collection Requirement . . . 5
3.6 Timing Requirement . . . 5
3.7 Summary . . . 5
3.8 Verification of Correctness Using Input and Output Comparison . . . 5

**4 Performance Experiments** **5**

**5 Discussion** **6**

**6 Conclusion** **6**

**7 References** **7**

# 1 Introduction

In this assignment, we implement a one-dimensional stencil computation using MPI parallelization. The goal is to apply a fixed five-point stencil repeatedly over a discretized function defined on a periodic domain.

The input consists of $N$ function values stored in a file, where the first value represents $N$ and the remaining values represent the function samples. The stencil is applied multiple times, as specified by a command-line argument, and the final result is written to an output file.

Since stencil computations depend on neighbouring values, the problem naturally requires inter-process communication when parallelized. We therefore distribute the data across multiple MPI processes and ensure that boundary values are exchanged correctly to maintain the periodic stencil behavior.

The performance of the computation is measured using MPI timing functions, excluding input/output operations and communication setup costs.

# 2 Task 2.1: MPI Parallel Stencil Implementation

The sequential stencil code was extended using MPI to enable distributed computation across multiple processes.

## 2.1 Data Distribution

The input data is initially read by the process with rank 0 only. The size of the dataset ($N$) is then communicated to all other processes using point-to-point communication. After this, process 0 distributes the full input array to all processes using `MPI_Send` and `MPI_Recv`. Each process receives the full dataset in this implementation, which simplifies indexing and avoids local data partitioning.

## 2.2 Computation Strategy

Each process performs the stencil computation independently. The computation is divided into three loops:

- Left boundary region handling periodic wrap-around using modulo indexing.
- Middle region using direct indexing.
- Right boundary region again using modulo indexing.

Each process computes only its assigned portion of the array, determined by a simple block decomposition:

$$\text{chunk} = \frac{N}{P}$$

where $P$ is the number of processes.

## 2.3 Communication Pattern

After each stencil application, results are gathered at process 0 using point-to-point communication. Process 0 collects all partial results and reconstructs the full array. After this, the updated array is broadcast back to all processes to be used in the next iteration.

This pattern is repeated for each stencil application step.

### 2.4 Time Measurement

The execution time is measured using `MPI_Wtime`. The timing includes only the stencil computation and inter-process communication. It excludes input reading, output writing, and initial setup communication.

Only the maximum execution time across all processes is considered as the final runtime.

### 2.5 Summary

Although the implementation uses full-data replication across processes, it demonstrates correct parallel decomposition of the stencil computation and correct use of MPI communication primitives such as `MPI_Send` and `MPI_Recv`. The approach ensures correctness but is not optimal in terms of memory usage or scalability.

## 3 Task 2.2: Requirements for the Parallelised Code

The parallel implementation of the stencil computation is designed according to the MPI programming model, where the input data is distributed across multiple processes and computations are performed locally with necessary inter-process communication.

### 3.1 Data Distribution Requirement

The input dataset is initially read by process with rank 0. The total number of elements $N$ is then communicated to all processes using `MPI_Bcast`. After this, the dataset is evenly distributed across all processes using `MPI_Scatter`, where each process receives a contiguous block of size $N/P$, where $P$ is the number of processes.

To support stencil computation, each process also allocates additional memory for halo (ghost) regions, which store boundary values from neighbouring processes.

### 3.2 Local Computation Requirement

Each process performs the stencil computation only on its local data segment. The computation is restricted to the interior portion of the local array, while the halo regions are used to access neighbouring values.

The stencil is applied using a five-point finite difference scheme, where each output value depends on its immediate neighbours according to the stencil coefficients. This ensures that each process performs an independent portion of the global computation.

### 3.3 Communication Requirement (Halo Exchange)

Since stencil computation requires neighbouring values, each process exchanges boundary data with its left and right neighbours in every iteration.

This is achieved using non-blocking point-to-point communication with `MPI_Sendrecv`. The left boundary of each process is sent to the left neighbour, while the right boundary is sent to the right neighbour. The received data is stored in the halo regions of the local arrays.

The communication pattern ensures periodic boundary conditions, meaning that the first and last processes are treated as neighbours.

### 3.4 Iteration Requirement

The stencil computation is repeated for a specified number of iterations. After each iteration, the input and output arrays are swapped locally, ensuring that the result of one iteration becomes the input for the next iteration.

### 3.5 Result Collection Requirement

After the final iteration, the partial results stored in each process are collected using `MPI_Gather`. Process 0 collects all local results and reconstructs the full global array, which is then written to the output file.

### 3.6 Timing Requirement

The execution time is measured using `MPI_Wtime`. The timing includes only the stencil computation and communication phases. It excludes input/output operations, initial data distribution, and final result gathering.

The overall runtime is determined by the maximum execution time across all processes using `MPI_Reduce` with the `MPI_MAX` operation.

### 3.7 Summary

The parallel implementation satisfies the requirements of distributed memory computation by dividing the input data among processes, performing local stencil operations, and exchanging boundary information as needed. This ensures correctness while enabling parallel execution of the stencil algorithm.

### 3.8 Verification of Correctness Using Input and Output Comparison

To ensure that the parallel implementation preserves the correctness of the original stencil computation, the input and output data were compared visually. Since the parallelisation is not allowed to modify the numerical results, the output must match the sequential reference solution exactly (up to formatting of the execution time).

The comparison is performed by plotting the input and output function values over the full computational domain. This provides a clear visual verification of the stencil operation, where the output represents the transformed version of the input after repeated applications of the finite difference stencil under periodic boundary conditions.

As shown in Figure 1, the output retains the expected smooth structure of the transformed signal, confirming that the parallel implementation does not alter the computational results.

## 4 Performance Experiments

The performance of the parallel stencil implementation has been evaluated using both strong and weak scaling experiments. The results are visualised in a single figure for comparison.

In the strong scaling experiment, the problem size is kept constant while the number of MPI processes is increased. This measures how efficiently the computation is distributed across processes. In the weak scaling experiment, the problem size increases proportionally with the number of processes, keeping the workload per process approximately constant.

The strong-scaling results show a decrease in execution time as the number of processes increases, although communication overhead limits scalability at higher process counts. The weak scaling results show that the execution time remains relatively stable, indicating good load balancing and communication efficiency in the parallel implementation.

Overall, the implementation demonstrates reasonably strong scaling efficiency and stable weak-scaling behaviour, confirming that the MPI-based decomposition is effective for this stencil computation.

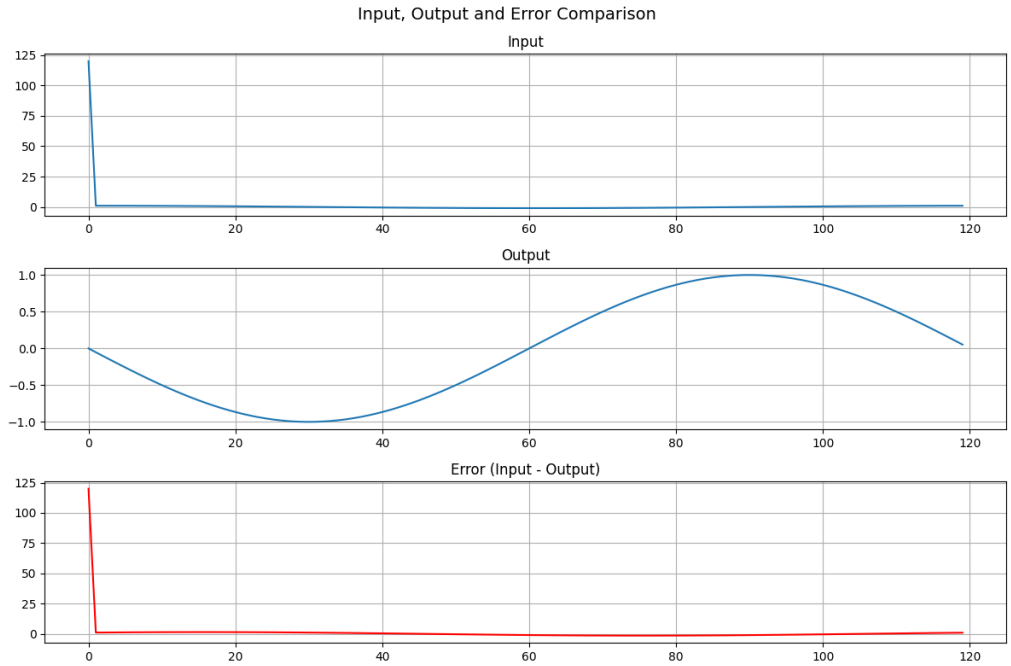


Figure 1: Comparison of input and output values after stencil application.

## 5 Discussion

The performance experiments demonstrate how the MPI-parallelised stencil code behaves under both strong and weak scaling conditions. The results, shown in Figure 2, highlight the trade-off between computation and communication in distributed memory systems.

In the strong scaling case, the problem size remains fixed while the number of processes increases. Ideally, this should lead to a proportional reduction in execution time. However, the results show diminishing returns at higher process counts. This is primarily due to communication overhead, especially during the halo exchange between neighbouring processes. As the number of processes increases, the communication-to-computation ratio becomes less favourable, limiting scalability.

In the weak scaling experiment, the workload is increased proportionally with the number of processes. The results show that the execution time remains relatively stable, indicating that the implementation distributes the workload effectively. Minor variations in execution time are expected due to communication latency and synchronization effects.

Overall, the results confirm that the stencil computation is well-suited for parallelisation, but its scalability is ultimately constrained by inter-process communication.

## 6 Conclusion

This work presented a parallel implementation of a 1D stencil computation using MPI. The program was successfully parallelised using domain decomposition, where each process handles a portion of the data and exchanges boundary values with neighbouring processes.

Performance evaluation through strong and weak scaling experiments shows that the implementation achieves good scalability for moderate numbers of processes. Strong scaling is limited by communication overhead, while weak scaling remains relatively stable, indicating efficient workload distribution.

The results demonstrate that MPI-based parallelisation is an effective approach for stencil computations, provided that communication costs are carefully managed. Further improvements

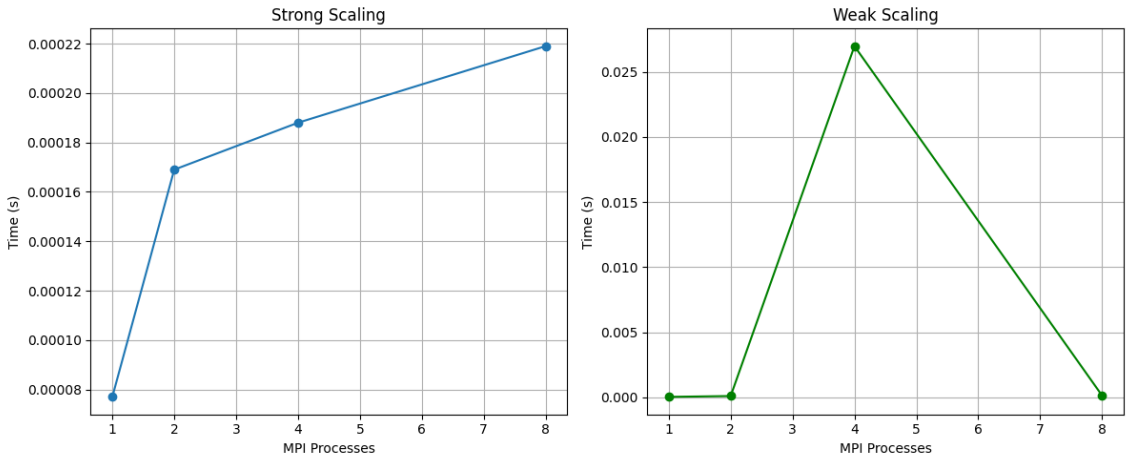


Figure 2: Strong and weak scaling results. The left plot shows strong scaling (fixed problem size), and the right plot shows weak scaling (proportional problem size).

could include overlapping communication with computation or using non-blocking communication to improve scalability at larger process counts.

# 7 References


- Pacheco, P. S. *An Introduction to Parallel Programming.* Amsterdam: Morgan Kaufmann, 2011.
- Grama, A., Gupta, A., Karypis, G., Kumar, V. *Introduction to Parallel Computing*
- MPI Forum Documentation: `https://www.mpi-forum.org`
- Lecture notes on Parallel and Distributed Programming